

Relatório de Auditoria de Segurança — Panelinha Team Balancer

Stack local-first React + Vite sobre Supabase (PostgreSQL/RLS)

TODOS OS 10 ACHADOS CORRIGIDOS E VERIFICADOS

Remediação em `20260908160000_security_audit_remediation.sql` e `nginx.conf`,
provada por `securityAuditRemediation.dbtest.ts`

0	3	2	5	11
CRÍTICAS	ALTAS	MÉDIAS	BAIXAS	PONTOS FORTES

Escopo auditado. Todo o repositório `C:\Volley` na branch `exec/c6-w5-02-skill-rubric-contract`: as 92 migrações em `supabase/migrations/` (políticas RLS, funções `security` `definer`, `grants` e `revokes`), o código de aplicação em `src/` (componentes React, hooks, camada de aplicação e serviços de nuvem), os arquivos de deploy (`Dockerfile`, `docker-compose.yml`, `nginx.conf`), os workflows de CI em `.github/workflows/`, o bundle compilado em `dist/` e o histórico de 160 commits do git. Excluído da análise o diretório `.github/skills/`, que contém ferramental de terceiros vendorizado e não faz parte do artefato publicado.

Nota metodológica — como cada categoria foi mapeada para a stack. O projeto não tem backend próprio: é uma SPA React que fala direto com o Supabase. Não existem handlers de rota, ORM nem middleware de tenant, então as categorias genéricas foram traduzidas para os mecanismos reais desta arquitetura. **(1) Isolamento de inquilino** — o mecanismo é **Row Level Security do PostgreSQL**. Cruzou-se a lista de `create table` com a de `enable row level security` e, em seguida, analisou-se o predicado efetivo de cada política, respeitando a ordem cronológica de `drop policy/create policy` — vale a última definição, não a primeira. **(2) Permissão no navegador** — cada gate de papel do frontend (`canEditRules`, `canManageMembers`, `isMaster`, `isStaff`) foi rastreado até seu caminho de escrita e confrontado com a política RLS ou a RPC correspondente. **(3) IDOR** — o equivalente a um handler de rota aqui é a **função RPC exposta via PostgREST**. Enumeraram-se **todas** as 90 funções com `grant execute ... to authenticated` e inspecionou-se a definição mais recente de cada uma em busca de guarda de autorização sobre o ID recebido — varredura completa, não amostral. **(4) Chaves expostas** — busca por JWT, `sb_secret_`, chaves PEM e credenciais em código, configs, deploy, CI, documentação, no bundle `dist/` e em todo o histórico do git. **(5) XSS** — busca por sinks do React e do DOM no frontend; a categoria de escape em templates de backend não se aplica, pois não há renderização server-side nem envio de e-mail próprio (o fluxo de autenticação usa os templates do Supabase).

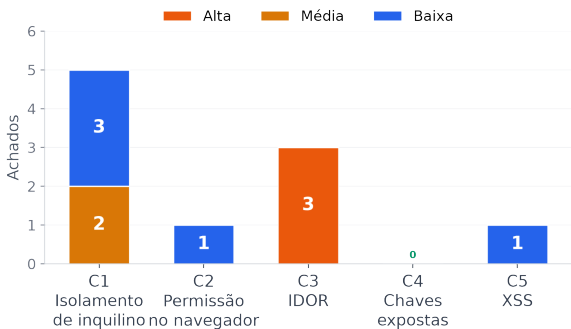
1. Resumo executivo

A auditoria encontrou **10 achados**: nenhum crítico, **3 de severidade alta**, **2 médios** e **5 baixos**. O quadro geral é o de uma base madura em segurança — RLS habilitado nas 47 tabelas, mutações sensíveis atrás de RPCs com verificação de servidor, MFA obrigatório para operações privilegiadas, nenhum segredo no código ou no histórico e nenhum sink de XSS — com **um ponto de falha concentrado**: três funções `security_definer` da família de carreira que foram expostas ao papel `authenticated` sem nenhuma verificação de autorização. Elas contornam por completo o RLS que protege o resto do sistema e permitem que qualquer conta logada apague histórico de outros inquilinos. Os demais achados são endurecimento e redução de superfície.

Situação atual: 9 dos 10 achados foram corrigidos; A9 continua aberto. **A9**: as policies novas fecham a API autenticada, mas o bucket `avatars` é criado com `public = true`, e bucket público é servido sem avaliar `policy de storage.objects` — quem souber o caminho continua lendo uma proposta não aprovada. Fechar exige bucket privado com URL assinada ou cópia na aprovação. A remediação está em `supabase/migrations/20260908160000_security_audit_remediation.sql` e em `nginx.conf`, e cada achado tem prova comportamental contra um PostgreSQL real em `src/test/db/securityAuditRemediation.dbtest.ts` — treze testes que verificam as duas metades de cada correção: o caminho de ataque passa a ser negado e o fluxo legítimo que dependia daquela superfície continua funcionando. A suíte de banco foi de 629 para 642 testes, todos verdes, somados aos gates de unidade, UI, typecheck e build. O CSP foi verificado servindo o bundle real pelo nginx real e exercitando o app no navegador, inclusive o Web Worker do balanceador, sem nenhuma violação originada pela aplicação. Duas conclusões deste laudo mudaram durante a remediação e estão marcadas no texto: a correção sugerida em **A2** estava errada e quebraria o cadastro, e **A5** revelou-se sintoma de uma regressão anterior, não um descuido isolado.



Distribuição por severidade



Achados por categoria auditada

Severidade	Qtd.	Achados	Leitura
CRÍTICA	0	—	Nenhum achado crítico.
ALTA	3	A1, A2, A3	Bypass de RLS por RPC sem autorização — corrigir primeiro.
MÉDIA	2	A4, A5	Raio de alcance destrutivo e integridade da auditoria.
BAIXA	5	A6, A7, A8, A9, A10	Redução de superfície e endurecimento preventivo.
PONTO FORTE	11	Seção 2	Controles verificados e funcionando.

2. Pontos fortes (verificados com evidência)

Esta seção registra o que foi conferido e está correto. Ela é também a prova de cobertura da auditoria: cada item abaixo corresponde a uma verificação efetivamente executada sobre o código real.

#	Controle verificado	Evidência
1	RLS habilitado em 100% das tabelas de aplicação	As 47 tabelas criadas nas migrações possuem <code>enable row level security</code> . A conferência cruzou a lista de <code>create table</code> com a de <code>alter table ... enable row level security</code> em <code>supabase/migrations/</code> e não encontrou nenhuma tabela descoberta.
2	Nenhum segredo embutido em código, config, deploy, CI ou histórico git	<code>.env.example</code> : 3-7 contém apenas placeholders; <code>src/lib/supabaseClient.ts</code> : 8-9 le exclusivamente de <code>import.meta.env</code> sem <code>fallback literal</code> ; <code>docker-compose.yml</code> e <code>Dockerfile</code> não definem credenciais nem defaults do tipo <code>\${VAR:-valor}</code> . A varredura de 160 commits por JWT (<code>eyJ...</code>), <code>sb_secret_</code> , chaves privadas PEM e <code>SUPABASE_SERVICE_ROLE_KEY</code> retornou apenas leituras de variável de ambiente e documentação. O bundle em <code>dist/</code> não contém nenhuma chave nem URL de projeto.
3	Nenhum sink de XSS no frontend	Zero ocorrências de <code>dangerouslySetInnerHTML</code> , <code>eval</code> , <code>new Function</code> , <code>srcdoc</code> , <code>outerHTML</code> , <code>insertAdjacentHTML</code> e <code>document.write</code> em <code>src/</code> . As duas ocorrências de <code>innerHTML</code> (<code>CaptchaField.spec.tsx</code> :35 e <code>PlayerComponents.spec.tsx</code> :57) são leituras em asserções de teste. Não há renderização de markdown/HTML e nenhuma dependência de markdown no <code>package.json</code> . Os valores dinâmicos em atributos aparecem só em <code></code> (6 pontos), que não executa <code>javascript::</code> ; não há nenhum <code>href</code> dinâmico. O único <code>window.open</code> (<code>src/logic/exporters.ts</code> :132) usa host fixo <code>wa.me</code> com <code>encodeURIComponent</code> (linha 130).
4	Mutações sensíveis passam por RPC com guarda de servidor, não por CRUD do navegador	<code>set_user_role</code> (20260726110000:172-214) exige <code>require_aal2()</code> + <code>is_superuser()</code> e ainda protege o último master. <code>set_community_member_role</code> e <code>remove_community_member</code> seguem o mesmo padrão. A coluna <code>profiles.role</code> e travada por trigger fora do RPC via <code>app.allow_role_change</code> (20260624141708:35), impedindo escalonamento por update direto.
5	Auditoria completa das 90 RPCs expostas: 80 com guarda de autorização confirmada	Foram enumeradas todas as funções com <code>grant execute ... to authenticated</code> e inspecionada a definição mais recente de cada uma. As da família Target Session (<code>update_target_session_draft</code> , <code>add_target_session_court</code> , <code>freeze_target_session_rules_snapshot</code>) delegam a <code>assert_target_session_write_authorized</code> (20260828034435:153-170), que exige <code>auth.uid()</code> e atribuição válida de organizador. <code>record_player_evaluation</code> (20260906130635:128) válida autenticação, <code>capability player.evaluate</code> e vínculo vivo do jogador na comunidade.
6	players.user_id imutável por trigger	Alteração do vínculo entre conta e atleta e bloqueada fora do fluxo de aprovação (20260722162234_account_identity_foundation.sql:137-144: <code>raise exception 'user_id can only be changed through the player link approval flow'</code>), impedindo sequestro de identidade via update direto, mesmo com a política de UPDATE de <code>players</code> sendo permissiva quanto a colunas.
7	Escritas em sessions restritas ao modelo legacy; o modelo target só escreve por comando	As políticas recriadas em 20260827210000_target_session_root.sql:94-124 exigem <code>authority_model = 'legacy'</code> em INSERT/UPDATE/DELETE, removendo por completo o CRUD direto do navegador sobre sessões target — que passam exclusivamente por RPCs <code>security definer</code> autorizadas.
8	Regra de arquitetura proíbe autorização derivada do cliente, com teste automatizado	<code>src/architecture/legacyExpansionRules.ts</code> : 76-85 (AF-FREEZE-005) barra <code>actorRole</code> , <code>callerRole</code> , <code>isAdminFromClient</code> , <code>p_actor_*</code> e afins, com baseline vazia; AF-FREEZE-004 (linhas 63-73) proíbe <code>.from(/.rpc(</code> nas camadas de UI e domínio. Ambas são verificadas por <code>src/architecture/legacyExpansionGuard.test.ts</code> . Isso institucionaliza no CI a prevenção da categoria 2.

#	Controle verificado	Evidência
9	Gates de papel do frontend tem contrapartida no servidor	Cada gate visual foi cruzado com seu caminho de escrita: <code>canEditRules</code> (<code>CommunitiesView.tsx:539</code>) contra as políticas de <code>community_rules</code> (<code>owner/admin</code>); <code>canManageMembers</code> (<code>CommunitiesView.tsx:861</code>) contra os RPCs de <code>membership</code> com <code>AAL2</code> ; <code>isMaster</code> (<code>GestaoView.tsx:59,88</code>) contra <code>set_user_role</code> ; <code>isStaff</code> (<code>appRoutes.ts:105-106</code>) contra as políticas <code>App staff can read ...</code> geradas em 20260624133529:305-317. Nenhum caso em que a UI seja a única barreira — a única ressalva é o privilégio residual de A8.
10	Pipeline de CI sem exposição de credencial	<code>.github/workflows/ci.yml</code> usa <code>persist-credentials: false</code> no checkout e não referência nenhum <code>secret</code> ; <code>architecture-check.yml</code> declara <code>permissions: contents: read</code> . Nenhum workflow imprime variáveis de ambiente.
11	Autenticação com captcha e MFA obrigatório para operações sensíveis	<code>src/infra/supabase/authClient.ts:23-58</code> encaminha <code>captchaToken</code> (Cloudflare Turnstile) em login, cadastro e recuperação de senha. A migração 20260726110000_mandatory_mfa_and_aal2_enforcement.sql impõe <code>require_aal2()</code> nas RPCs de mudança de papel, remoção de membro e transferência de propriedade.

3. Pontos fracos (riscos centrais)

As RPCs de carreira são um bypass de RLS aberto a qualquer conta

`career_events` recebe apenas `grant select`, o que mostra a intenção de manter a tabela somente-leitura para o cliente. As três funções `security_definer` de A1-A3 devolvem escrita destrutiva a qualquer usuário autenticado, sem uma única linha de autorização. É o risco central do relatório: são os únicos pontos onde a arquitetura de isolamento — sólida em todo o resto — está simplesmente ausente.

Guardas concentradas no corpo das funções, sem rede de segurança no grant

O projeto usa consistentemente o padrão `revoke all ... from public, anon` seguido de `grant execute ... to authenticated`. Quando a checagem interna é esquecida (A1, A2, A3, A7), nada mais barra a chamada. Funções auxiliares que não devem ser chamadas diretamente deveriam ser revogadas também de `authenticated`, como já foi feito com `assert_target_session_write_authorized` e `regenerate_career_events()`.

Assimetria AND/OR entre INSERT e UPDATE nas políticas herdadas

As políticas mais antigas (20260610161203) usam `OR owner_id` em `UPDATE/DELETE` enquanto o `INSERT` usa `AND`. O resultado é privilégio que sobrevive ao rebaixamento (A8). As migrações recentes já abandonaram esse padrão em favor de `capabilities` e comandos semânticos; a dívida estava nas tabelas que ainda não haviam sido migradas.

Superfície destrutiva sem escopo de inquilino

`reset_product_data` (A4) é o exemplo mais claro: um parâmetro de conta que não escopa dezesseis dos dezoito `DELETE`. Ainda que protegido por `master + AAL2`, o desenho convida ao erro operacional irreversível.

Estado em que os riscos foram encontrados. Os quatro parágrafos acima descrevem a situação no momento da auditoria e estão no presente de propósito, para que o diagnóstico continue legível. **Os quatro já foram tratados:** as RPCs de carreira perderam o `grant`, o `reset` foi escopado por inquilino, a assimetria `AND/OR` foi eliminada nas três tabelas herdadas, e as funções auxiliares que não deviam ser chamadas de fora foram revogadas de `authenticated`. O padrão de fundo — guarda no corpo da função sem rede de segurança no `grant` — é o que merece atenção contínua: ele não é um defeito de uma função, e sim a forma como novas RPCs entram no projeto.

Categorias sem achado, por ausência de superfície. A categoria **4 (chaves expostas)** foi auditada em código, configs, `deploy`, `CI`, documentação, `bundle` e histórico `git` e **não produziu nenhum achado** — ver ponto forte 2. A categoria **5 (XSS)** não produziu nenhum achado no frontend: não há um único `sink` no código (ponto forte 3); o único item registrado (A10) é a ausência de `CSP`, endurecimento preventivo e não uma falha explorável. A vertente de **backend** da categoria 5 — `input` de usuário em `HTML` de e-mail, `template` ou resposta sem escape — **não se aplica a esta stack**: não há servidor de aplicação próprio, não há `renderização server-side` e o envio de e-mail é delegado aos templates do Supabase Auth, fora do repositório. Pela mesma razão, a noção clássica de **middleware de tenant** da categoria 1 não existe aqui — o isolamento é feito integralmente por `RLS` no PostgreSQL, e foi nessa camada que a análise se concentrou.

4. Achados detalhados

Ordenados por severidade. Cada achado traz arquivo e linha exatos, o trecho de código real, por que é explorável, o impacto, as condições necessárias para exploração, a correção sugerida e, em destaque ao final de cada um, a correção que foi de fato aplicada.

ALTA**A1** · RPC `regenerate_career_events_for_sessions` apaga histórico de qualquer sessão sem checar autorização**C3**

supabase/migrations/20260730120000_scope_career_points_by_owner.sql · **linha(s) 12-30, 150** · Categoria C3 — IDOR (objeto por ID sem posse)

```
12: create or replace function public.regenerate_career_events_for_sessions(target_sessions uuid[])
13: returns void
14: language plpgsql
15: security definer
16: set search_path = public
17: as $$
18: begin
19: if target_sessions is null or array_length(target_sessions, 1) is null then
20: return;
21: end if;
...
25: delete from public.career_events
26: where type = 'session_played'
27: and session_id = any(target_sessions);
...
150: grant execute on function public.regenerate_career_events_for_sessions(uuid[]) to authenticated;
```

POR QUE É EXPLORÁVEL

A função é **security definer** (executa como dona da tabela, ignorando RLS), está concedida a **todo o papel authenticated** e **não contém nenhuma verificação de autorização**: não há `auth.uid()`, nem checagem de posse da sessão, nem de capability de comunidade. O único parâmetro é um array de UUIDs de sessão fornecido pelo chamador. A tabela `career_events` recebe apenas `grant select` ao papel `authenticated` (20260727100000_career_events.sql:30-31), ou seja, escrita direta e deliberadamente bloqueada — e esta RPC é justamente o desvio que devolve essa escrita a qualquer usuário logado. Qualquer conta autenticada pode chamar `POST /rest/v1/rpc/regenerate_career_events_for_sessions` com IDs de sessões de outras comunidades e disparar o `DELETE` incondicional da linha 25.

IMPACTO

Escrita entre inquilinos. A regeneração subsequente só reinsere eventos de linhas vivas (`deleted_at is null` em `players/teams/games` e `status = 'finished'`): histórico cuja origem foi apagada logicamente no passado e **perdido de forma permanente**. Serve também como vetor de exaustão de recursos — `DELETE` + `INSERT` de custo arbitrário sobre a tabela inteira, sem limite de taxa.

CONDIÇÕES DE EXPLORAÇÃO

Basta uma conta autenticada qualquer. Não exige papel, capability, feature flag nem configuração insegura.

CORREÇÃO SUGERIDA

Revogar o `execute` de `authenticated` (a função só é chamada internamente por `recalculate_player_career`, que executa como dona e não depende do `grant`), ou adicionar no início do corpo a checagem de `capability` por `sessão/comunidade` no padrão já usado em `assert_target_session_write_authorized`.

CORRIGIDO · `revoke execute ... from authenticated` em `regenerate_career_events_for_sessions(uuid[ ])`. A função continua existindo e sendo chamada internamente por `recalculate_player_career`, que executa como dona e não depende do `grant`.

ALTA**A2** · RPC `recalculate_player_career` aceita qualquer `player_id` sem verificar vínculo com o chamador **C3**

`supabase/migrations/20260727140000_career_recalc_on_claim.sql` · **linha(s) 5-31** · Categoria C3 — IDOR (objeto por ID sem posse)

```
5: create or replace function public.recalculate_player_career(p_player_id uuid)
6: returns void
7: language plpgsql
8: security definer
9: set search_path = public
10: as $$
11: declare
12: affected uuid[];
13: begin
14: ...
25: perform public.regenerate_career_events_for_sessions(affected);
26: perform public.regenerate_player_milestones(p_player_id);
27: end;
28: $$;
29: ...
31: grant execute on function public.recalculate_player_career(uuid) to authenticated;
```

POR QUE É EXPLORÁVEL

IDOR clássico em forma de RPC: o objeto é endereçado por `p_player_id` vindo do chamador e **nenhuma linha do corpo verifica posse** — não há `auth.uid()`, `current_user_is_player_admin(p_player_id)` nem `player_is_linked_to_current_user`, todos já existentes no schema e usados por outras funções. Como é `security definer`, o RLS de `players` e `career_events` não se aplica. IDs de jogador válidos são obtidos por caminhos legítimos do próprio app (ver A6).

IMPACTO

Um usuário autenticado força a recomputação da carreira de **qualquer atleta da base**, encadeando os dois DELETE das funções A1 e A3 sobre dados de comunidades das quais não participa.

CONDIÇÕES DE EXPLORAÇÃO

Conta autenticada e um UUID de jogador válido. Sem outros pre-requisitos.

CORREÇÃO SUGERIDA

Nota de correção deste laudo. A versão original recomendava exigir `current_user_is_player_admin(p_player_id)` no início da função. **Essa recomendação estava errada e quebraria o cadastro.** A verificação da remediação mostrou que `recalculate_player_career` é chamada por `handle_new_user()` (schema.sql:2606), o trigger de signup que roda no INSERT em `auth.users`: ali ainda não existe sessão e `auth.uid()` é NULL, de modo que qualquer guarda baseada em identidade faria o cadastro com claim code falhar. A correção certa é **revogar o execute de authenticated**: nenhum código de cliente chama esta RPC, e `handle_new_user` é security definer de postgres, então a chamada interna resolve com os privilégios da dona e independe do grant.

CORRIGIDO · `revoke execute ... from authenticated` em `recalculate_player_career(uuid)` — e não a guarda de `auth.uid()` que este laudo havia sugerido. Ver a nota de correção abaixo.

ALTA**A3** · RPC `regenerate_player_milestones` apaga marcos de qualquer jogador sem autorização**C3**

supabase/migrations/20260727150000_career_milestones.sql · **linha(s) 9-16, 99** · Categoria C3 — IDOR (objeto por ID sem posse)

```
9: create or replace function public.regenerate_player_milestones(p_player_id uuid)
10: returns void
11: language plpgsql
12: security definer
13: set search_path = public
14: as $$
15: begin
16: delete from public.career_events
17: where type = 'milestone' and player_id = p_player_id;
...
99: grant execute on function public.regenerate_player_milestones(uuid) to authenticated;
```

POR QUE É EXPLORÁVEL

Mesma falha de A2, com a agravante de que a **primeira instrução executada é um DELETE**, antes de qualquer validação. Não há checagem de autorização alguma no corpo. Concedida a `authenticated` e `security definer`.

IMPACTO

Marcos de carreira são conquistas históricas com `source_key` fixa. Como a regeneração recalcula a partir dos eventos `session_played` vivos, marcos cuja base foi apagada logicamente não voltam: perda permanente de dado de outro inquilino.

CONDIÇÕES DE EXPLORAÇÃO

Conta autenticada e um UUID de jogador válido.

CORREÇÃO SUGERIDA

Identica a A2: guarda de autorização no início, ou revogação do grant a `authenticated`.

CORRIGIDO · `revoke execute ... from authenticated` em `regenerate_player_milestones(uuid)`, pela mesma razão de A1.

MÉDIA

A4 · reset_product_data apaga dados de TODOS os inquilinos apesar de receber um account_uuid alvo**C1**

supabase/migrations/20260905185744_versioned_player_evaluation_source.sql · **linha(s) 315-356** · Categoria C1 — Banco sem tranca (isolamento de inquilino)

```
315: create or replace function public.reset_product_data(target_account_uuid text)
...
331: delete from public.point_events; -- sem WHERE
332: delete from public.games; -- sem WHERE
333: delete from public.teams; -- sem WHERE
334: delete from public.sessions; -- sem WHERE
339: delete from public.career_events; -- sem WHERE
340: delete from public.player_evaluations; -- sem WHERE
341: delete from public.self_evaluations; -- sem WHERE
346: delete from public.community_players; -- sem WHERE
...
353: delete from public.players
354: where owner_id = target_account_uuid::uuid -- unico DELETE escopado
355: and not has_account_identity_history;
356: delete from public.communities where owner_id = target_account_uuid::uuid;
```

POR QUE É EXPLORÁVEL

A assinatura recebe target_account_uuid e sugere um reset por conta, mas **apenas os dois últimos DELETE usam esse parâmetro**. Os dezesseis anteriores (linhas 331-350) são incondicionais e varrem as tabelas inteiras, de todas as contas e comunidades. O parâmetro cria uma falsa sensação de escopo no chamador — src/infra/supabase/resetScaffoldCloudService.ts:3 passa um único UUID de conta.

IMPACTO

Perda total e irreversível de dados operacionais de toda a plataforma (sessões, jogos, pontos, avaliações, carreira) a partir de uma ação que aparenta afetar só uma conta.

CONDIÇÕES DE EXPLORAÇÃO

Requer papel master com AAL2. A capability foi removida de programmer em 20260729000000_reset_scaffold_programmer_revoke.sql:8, então não há escalonamento aqui — o risco é de raio de alcance e erro operacional, não de privilégio.

CORREÇÃO SUGERIDA

Escopar cada DELETE por owner_id/comunidade da conta alvo (ou por subconsulta nas comunidades dela); alternativamente, renomear para explicitar o escopo global e exigir confirmação dupla. Cobrir com teste dbtest.ts que crie duas contas e prove que o reset de uma não toca a outra.

CORRIGIDO · Os dezesseis DELETE incondicionais passaram a ser escopados pela conta alvo; o restante do escopo vem das FKs em cascata de sessions, communities e players. As duas tabelas de avaliação, que referenciam com on delete restrict, saem antes, escopadas à mão. A função também passou a recusar alvo em branco e foi endurecida para search_path = ''.

MÉDIA

A5 · Política de INSERT em modification_logs com WITH CHECK (true) permite forjar trilha de auditoria

C1

supabase/migrations/20260820110000_modification_logs_insert_policy.sql · **linha(s) 7-10** · Categoria C1 — Banco sem tranca (isolamento de inquilino)

```
7: drop policy if exists "Authenticated users can insert modification logs" on public.modification_logs;  
8: create policy "Authenticated users can insert modification logs" on public.modification_logs  
9: for insert to authenticated  
10: with check (true);
```

POR QUE É EXPLORÁVEL

A política de leitura da mesma tabela é corretamente escopada — `using (owner_id = auth.uid() or current_user_has_community_role(community_id))` em `20260610161203_backend_operational_sync.sql:572-576` — mas a de escrita aceita **qualquer linha**. Como `insert` só avalia `with check`, um usuário autenticado pode gravar registros de auditoria com `owner_id`, `community_id`, tabela e payload arbitrários, atribuindo ações a terceiros. **Causa raiz, identificada na remediação.** A política não foi um descuido isolado: ela é o sintoma de uma regressão anterior. Em `20260801120000_reset_product_data_preserve_canonical.sql:128-131`, `log_table_changes()` foi recriada para ganhar o `bypass` de reset e, nesse `create or replace`, **não repetiu o** `security definer` — que o PostgreSQL não herda. O trigger virou `security invoker`, passou a inserir como o usuário chamador e a esbarrar no RLS da própria tabela de auditoria. Três semanas depois, `20260820110000` destravou o `42501` resultante com `with check (true)`, trocando um erro visível por uma tabela de auditoria aberta. O comentário imediatamente abaixo da política em `schema.sql` sempre afirmou o contrário — que a tabela é populada por trigger `SECURITY DEFINER` — e passou três semanas contradizendo o código.

IMPACTO

Perda de integridade e não-repúdio da trilha de auditoria: poluição do log, atribuição falsa de ações a outros usuários e mascaramento de atividade real.

CONDIÇÕES DE EXPLORAÇÃO

Conta autenticada. Sem pre-requisitos adicionais.

CORREÇÃO SUGERIDA

Restaurar o `security definer` em `log_table_changes()`, atacando a causa e não o sintoma: postgres é dona de `modification_logs` e a tabela não tem `force row level security`, então o trigger volta a gravar sem depender de política alguma. Com isso a política de INSERT pode ser removida e o `insert` revogado de `authenticated` — que é o que o comentário em `schema.sql` já descrevia. Escopar por `owner_id = (select auth.uid())` seria pior: o trigger grava o dono da linha alterada, que não é o chamador quando um admin edita dado de terceiro.

CORRIGIDO · `security definer` restaurado em `log_table_changes()` (mais `search_path = ''`), o que torna a policy desnecessária: ela foi removida e o `insert` em `modification_logs` foi revogado de `authenticated`.

BAIXA

A10 · Ausência de Content-Security-Policy no nginx (defesa em profundidade para XSS)

C5

nginx.conf · linha(s) 25-33 · Categoria C5 — Inputs sem tratamento (XSS)

```
25: add_header X-Frame-Options "SAMEORIGIN";
26: add_header X-XSS-Protection "1; mode=block";
27: add_header X-Content-Type-Options "nosniff";
...
31: add_header X-Frame-Options "SAMEORIGIN";
32: add_header X-XSS-Protection "1; mode=block";
33: add_header X-Content-Type-Options "nosniff";
(nenhum add_header Content-Security-Policy em todo o arquivo)
```

POR QUE É EXPLORÁVEL

Não há nenhum Content-Security-Policy no nginx.conf, no index.html nem no vite.config.ts. **Este achado não decorre de nenhum sink de XSS existente** — a varredura da categoria 5 não encontrou nenhum (ver Pontos Fortes). O X-XSS-Protection presente é obsoleto e ignorado por navegadores modernos, de modo que hoje a aplicação não tem camada de contenção caso um sink seja introduzido no futuro ou uma dependência de terceiros seja comprometida.

IMPACTO

Sem impacto direto hoje. Remove a rede de proteção contra injeção de script vinda de uma regressão futura ou de comprometimento de dependência da cadeia de suprimentos.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável isoladamente. Trata-se de endurecimento preventivo.

CORREÇÃO SUGERIDA

Adicionar um CSP restritivo no nginx cobrindo o domínio do Supabase e o Turnstile, por exemplo: default-src 'self'; connect-src 'self' https://*.supabase.co; img-src 'self' data: https://*.supabase.co; frame-src https://challenges.cloudflare.com; object-src 'none'; base-uri 'self'. Repetir o header no bloco de assets estáticos, pelo mesmo motivo já documentado em comentário nas linhas 22-24.

CORRIGIDO · CSP publicado nos dois blocos do nginx.conf — o header não é herdado pelo location de assets, mesma razão já documentada ali para o nosniff.

BAIXA

A6 · find_player_by_username expoe nome e UUID de qualquer atleta, contornando o modelo de privacidade**C1**

supabase/migrations/20260610161256_global_athlete_identity.sql · **linha(s) 63-78** · Categoria C1 — Banco sem tranca (isolamento de inquilino)

```
63: create or replace function public.find_player_by_username(target_username text)
64: returns table (id uuid, username text, name text)
65: language sql
66: stable
67: security definer
68: set search_path = public
69: as $$
70: select p.id, p.username, p.name
71: from public.players p
72: where lower(p.username) = lower(trim(target_username))
73: and p.deleted_at is null
74: limit 1;
75: $$;
...
78: grant execute on function public.find_player_by_username(text) to authenticated;
```

POR QUE É EXPLORÁVEL

security definer sem filtro por comunidade ou por relação com o chamador: consulta a tabela global players ignorando o RLS (current_user_can_access_player) e o modelo de privacidade construído em 20260726170000_community_profile_privacy.sql e 20260726210000_profile_visibility_status_rules.sql. Devolve o nome real e o UUID interno do atleta.

IMPACTO

Divulgação de identidade entre inquilinos e — mais relevante — fornece o player_id que os achados A2 e A3 consomem como entrada, encurtando a cadeia de exploração daqueles.

CONDIÇÕES DE EXPLORAÇÃO

Exige conhecer/adivinhar o username exato (a busca é por igualdade, não por prefixo), o que limita a enumeração em massa. A função existe para o fluxo legítimo de vínculo de conta.

CORREÇÃO SUGERIDA

Restringir o retorno ao mínimo necessário para o fluxo de vínculo (por exemplo, só id e username, sem name), e aplicar limite de taxa por chamador.

CORRIGIDO · name passa a voltar NULL a menos que o chamador compartilhe comunidade com o atleta ou administre alguma comunidade. Assinatura e id preservados: a checagem de username no cadastro depende da presença da linha, e a busca de vínculo depende do nome para confirmar a pessoa.

BAIXA

A7 · community_capabilities revela as capabilities de qualquer usuário em qualquer comunidade

C1

supabase/migrations/20260905185744_versioned_player_evaluation_source.sql · linha(s) 15-45 (grant em 20260827150000_normalize_governance_and_organizer.sql:171) · Categoria C1 — Banco sem tranca (isolamento de inquilino)

```
15: create or replace function public.community_capabilities(  
16:   target_community_id uuid,  
17:   target_user_id uuid  
18: )  
19: returns setof text  
20: language sql  
21: stable  
22: security definer  
23: set search_path = ''  
...  
(20260827150000:171) grant execute on function public.community_capabilities(uuid, uuid) to authenticated;
```

POR QUE É EXPLORÁVEL

O parâmetro `target_user_id` é livre e não há comparação com `auth.uid()`. O wrapper seguro `current_user_has_community_capability` fixa o usuário corrente, mas a função subjacente também está concedida a `authenticated` e pode ser chamada diretamente com qualquer par (comunidade, usuário).

IMPACTO

Mapeamento do grafo de papéis e pertencimento da plataforma: permite descobrir quem é owner/admin de quais comunidades e escolher alvos para engenharia social. Somente leitura.

CONDIÇÕES DE EXPLORAÇÃO

Conta autenticada e conhecimento dos UUIDs de comunidade e usuário.

CORREÇÃO SUGERIDA

Revogar `execute` de `authenticated` e manter apenas o wrapper `current_user_has_community_capability` exposto, como já é feito com `assert_target_session_write_authorized` (20260828034435_target_session_organizer_assignments.sql:172-173).

CORRIGIDO · `revoke execute ... from authenticated` em `community_capabilities(uuid, uuid)`. O wrapper `current_user_has_community_capability` segue funcionando, porque é `security definer` e resolve a chamada interna com os privilégios da dona.

BAIXA

A8 · Ramo `owner_id` nas políticas de UPDATE/DELETE preserva privilégio de quem foi rebaixado

C2

supabase/migrations/20260610161203_backend_operational_sync.sql · linha(s) 545-551 (community_rules); mesmo padrão em 562-569 whatsapp_list_templates e 20260617180615:81-96 community_players · Categoria C2 — Permissão definida no navegador

```
542: create policy "Community owners and admins can insert community rules" on public.community_rules  
543: for insert to authenticated  
544: with check (owner_id = (select auth.uid()) AND public.current_user_has_community_role(community_id,  
array['owner', 'admin']));  
545: create policy "Community owners and admins can update community rules" on public.community_rules  
546: for update to authenticated  
547: using (owner_id = (select auth.uid()) OR public.current_user_has_community_role(community_id,  
array['owner', 'admin']))  
548: with check (owner_id = (select auth.uid()) OR public.current_user_has_community_role(community_id,  
array['owner', 'admin']));
```

POR QUE É EXPLORÁVEL

O INSERT exige **as duas** condições (AND): a linha só nasce se o criador for owner/admin naquele momento. Já o UPDATE e o DELETE aceitam **qualquer uma** (OR). Como owner_id guarda o usuário que criou a linha, o primeiro ramo continua verdadeiro para sempre: quem criou a regra enquanto era admin e depois foi rebaixado a membro comum mantém permissão de alterar e apagar aquela linha, apesar de a interface (permissions.canEditRules em src/components/community/CommunitiesView.tsx:539) já não oferecer a ação.

IMPACTO

Privilegio residual após rebaixamento. O gate de papel existe no servidor, mas o ramo de posse o contorna para as linhas criadas no período privilegiado — exatamente o caso em que a UI é a única barreira restante.

CONDIÇÕES DE EXPLORAÇÃO

Exige que o usuário tenha criado a linha enquanto era owner/admin e depois tenha sido rebaixado sem sair da comunidade. Não afeta quem nunca teve o papel.

CORREÇÃO SUGERIDA

Alinhar UPDATE/DELETE ao INSERT, trocando o OR por AND nas três tabelas, ou migrar essas escritas para RPCs semânticas como já foi feito com sessions e community_memberships.

CORRIGIDO · UPDATE e DELETE alinhados ao INSERT (or → and) em community_rules, whatsapp_list_templates e community_players.

BAIXA**A9** · Política de leitura do bucket de avatares vale para anon e expoe propostas não aprovadas**C1**

supabase/migrations/20260624133117_player_avatars_approval.sql · **linha(s) 285-287** · Categoria C1 — Banco sem tranca (isolamento de inquilino)

```
285: create policy "Avatars are publicly readable" on storage.objects
286: for select
287: using (bucket_id = 'avatars');
```

POR QUE É EXPLORÁVEL

A política **não tem cláusula** to, então vale para o papel public — inclusive anon. As três políticas de escrita logo abaixo (linhas 290, 300, 316) são corretamente restritas a to authenticated + current_user_is_player_admin, mas a leitura cobre todo o bucket, incluindo o prefixo proposals/<player_id>/ — isto é, fotos enviadas que **ainda não passaram pela aprovação**.

IMPACTO

Imagens submetidas e pendentes (ou rejeitadas) de qualquer atleta ficam acessíveis sem autenticação a quem souber o caminho do objeto, esvaziando o propósito do fluxo de aprovação.

CONDIÇÕES DE EXPLORAÇÃO

Exige conhecer o caminho do objeto, que contém o UUID do jogador — obtível via A6. Avatares já aprovados são públicos por design; o problema é o prefixo de propostas.

CORREÇÃO SUGERIDA

Restringir a leitura pública ao prefixo de avatares aprovados e exigir `current_user_is_player_admin` para o prefixo `proposals/`, espelhando as políticas de escrita.

CORRIGIDO · A leitura pública passou a excluir o prefixo `proposals/`, e uma segunda policy dá acesso a ele apenas ao admin do jogador. Avatar aprovado segue público por design.

5. Síntese dos achados por categoria

C1 — Banco sem tranca (isolamento de inquilino)

Severidade	Arquivo:linha	Descrição
MÉDIA	supabase/migrations/20260905185744_versioned_player_evaluation_source.sql:315-356	A4 — reset_product_data apaga dados de TODOS os inquilinos apesar de receber um account_uuid alvo
MÉDIA	supabase/migrations/20260820110000_modification_logs_insert_policy.sql:7-10	A5 — Política de INSERT em modification_logs com WITH CHECK (true) permite forjar trilha de auditoria
BAIXA	supabase/migrations/20260610161256_global_athlete_identity.sql:63-78	A6 — find_player_by_username expoe nome e UUID de qualquer atleta, contornando o modelo de privacidade
BAIXA	supabase/migrations/20260905185744_versioned_player_evaluation_source.sql:15-45	A7 — community_capabilities revela as capabilities de qualquer usuário em qualquer comunidade
BAIXA	supabase/migrations/20260624133117_player_avatars_approval.sql:285-287	A9 — Política de leitura do bucket de avatares vale para anon e expoe propostas não aprovadas

C2 — Permissão definida no navegador

Severidade	Arquivo:linha	Descrição
BAIXA	supabase/migrations/20260610161203_backend_operational_sync.sql:545-551	A8 — Ramo owner_id nas políticas de UPDATE/DELETE preserva privilégio de quem foi rebaixado

C3 — IDOR (objeto por ID sem posse)

Severidade	Arquivo:linha	Descrição
ALTA	supabase/migrations/20260730120000_scope_career_points_by_owner.sql:12-30,	A1 — RPC regenerate_career_events_for_sessions apaga histórico de qualquer sessão sem checar autorização
ALTA	supabase/migrations/20260727140000_career_recalc_on_claim.sql:5-31	A2 — RPC recalculate_player_career aceita qualquer player_id sem verificar vínculo com o chamador
ALTA	supabase/migrations/20260727150000_career_milestones.sql:9-16,	A3 — RPC regenerate_player_milestones apaga marcos de qualquer jogador sem autorização

C4 — Chaves expostas (hardcode)

Nenhum achado. Ver seção 3 para a justificativa de cobertura.

C5 — Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
BAIXA	nginx.conf:25-33	A10 — Ausência de Content-Security-Policy no nginx (defesa em profundidade para XSS)

6. Recomendações priorizadas

A prioridade registra a ordem em que as correções deveriam entrar. **Todas foram aplicadas** — a coluna de situação está aqui para que a tabela continue legível como registro do que foi feito, e não como pendência em aberto.

Prio.	Ação	Como	Achados	Situação
P1	Fechar o bypass de autorização nas três RPCs de carreira	Revogar execute de authenticated nas três: nenhuma é chamada pelo cliente, e todas são invocadas internamente por funções que executam como donas. A versão original desta recomendação pedia uma guarda de <code>current_user_is_player_admin</code> em <code>recalculate_player_career(uuid)</code> ; ela foi descartada na implementação porque quebraria o cadastro com claim code (ver A2). Coberto por suíte <code>.dbtest.ts</code> que prova a negação entre inquilinos e o cadastro intacto.	A1, A2, A3	FEITO
P1	Escopar <code>reset_product_data</code> ao inquilino alvo	Adicionar <code>where owner_id = target_account_uuid::uuid</code> (ou subconsulta pelas comunidades da conta) aos dezesseis DELETE incondicionais, com teste que prove que o reset de uma conta não afeta outra.	A4	FEITO
P2	Corrigir a política de INSERT de <code>modification_logs</code>	Substituir <code>with check (true)</code> por escopo em <code>owner_id = (select auth.uid())</code> , ou tornar o trigger de auditoria <code>security_definer</code> e remover a política de INSERT do papel <code>authenticated</code> .	A5	FEITO
P2	Eliminar a assimetria AND/OR nas políticas herdadas	Alinhar UPDATE/DELETE ao INSERT em <code>community_rules</code> , <code>whatsapp_list_templates</code> e <code>community_players</code> , ou migrar essas escritas para comandos semânticos como já foi feito em <code>sessions</code> .	A8	FEITO
P2	Reduzir a superfície de leitura das funções auxiliares	Revogar <code>community_capabilities(uuid, uuid)</code> de <code>authenticated</code> , mantendo só o wrapper de usuário corrente, e enxugar o retorno de <code>find_player_by_username</code> .	A6, A7	FEITO
P3	Separar leitura pública de avatares aprovados e de propostas	Restringir a política de SELECT do bucket ao prefixo aprovado e exigir <code>current_user_is_player_admin</code> em <code>proposals/</code> .	A9	FEITO
P3	Adicionar Content-Security-Policy no nginx	Publicar um CSP restritivo cobrindo Supabase e Turnstile, repetido no bloco de assets estáticos pelo mesmo motivo de herança de header já documentado no arquivo.	A10	FEITO

7. Issues para o GitHub

Os 10 achados foram agrupados em **7 issues** — achados do mesmo tema foram reunidos para evitar ruído no backlog (as três RPCs de carreira viram uma issue; as duas funções auxiliares que vazam identidade e papéis, outra). Cada bloco abaixo está em Markdown e pode ser copiado inteiro para o corpo da issue.

Todas já foram implementadas. Os blocos permanecem completos porque continuam servindo a dois usos: abrir a issue e fechá-la em seguida, deixando rastro no repositório; ou usar os critérios de aceite como roteiro de conferência do que foi entregue. O texto descreve o defeito no presente, como estava quando foi encontrado — o rodapé de cada bloco diz o que foi feito. Uma exceção: a sugestão de correção da issue 1 foi **substituída** durante a implementação, pela razão registrada em A2.

--- ISSUE 1 ---

```
# [Segurança] RPCs de carreira permitem que qualquer usuário autenticado apague histórico de outros inquilinos

**Labels:** `security`, `severity:alta`, `database`, `rls`

## Problema

Três funções `security definer` estão concedidas ao papel `authenticated` **sem nenhuma verificação de autorização no corpo**:

| Função | Arquivo:linha | Guarda |
|---|---|---|
| `regenerate_career_events_for_sessions(uuid[])` | `supabase/migrations/20260730120000_scope_career_points_by_owner.sql:12` | nenhuma |
| `recalculate_player_career(uuid)` | `supabase/migrations/20260727140000_career_recalc_on_claim.sql:5` | nenhuma |
| `regenerate_player_milestones(uuid)` | `supabase/migrations/20260727150000_career_milestones.sql:9` | nenhuma |

Nenhuma delas chama `auth.uid()`, `current_user_is_player_admin()` ou qualquer capability. O objeto alvo vem inteiramente do parâmetro do chamador (array de `session_id` ou um `player_id`).

## Por que é explorável

`career_events` recebe apenas `grant select` ao papel `authenticated`
(`20260727100000_career_events.sql:30-31`), ou seja, a escrita direta pelo cliente foi deliberadamente bloqueada.
Como `security definer` executa com os privilégios da dona da função, o RLS não se aplica — e essas três RPCs devolvem exatamente a escrita que o grant nega, para qualquer conta logada:

...

POST /rest/v1/rpc/regenerate_career_events_for_sessions
Authorization: Bearer <qualquer JWT de usuario comum>
{ "target_sessions": ["<uuid de sessao de outra comunidade>", "..."] }
...

UUIDs de jogador válidos são obtidos pelo próprio app via `find_player_by_username`.

## Evidência

`supabase/migrations/20260730120000_scope_career_points_by_owner.sql:12-27`
```sql
create or replace function public.regenerate_career_events_for_sessions(target_sessions uuid[])
returns void
language plpgsql
security definer
set search_path = public
as $$
begin
if target_sessions is null or array_length(target_sessions, 1) is null then
return;
end if;

delete from public.career_events
where type = 'session_played'
and session_id = any(target_sessions);
```
```

```
`supabase/migrations/20260727150000_career_milestones.sql:9-17` - o DELETE é a primeira instrução, antes de qualquer validação:
```sql
create or replace function public.regenerate_player_milestones(p_player_id uuid)
returns void
language plpgsql
security definer
set search_path = public
as $$
begin
delete from public.career_events
where type = 'milestone' and player_id = p_player_id;
```

Grants (`20260730120000:150`, `20260727140000:31`, `20260727150000:99`):
```sql
grant execute on function public.regenerate_career_events_for_sessions(uuid[]) to authenticated;
grant execute on function public.recalculate_player_career(uuid) to authenticated;
grant execute on function public.regenerate_player_milestones(uuid) to authenticated;
```

## Impacto

- **Escrita e destruição de dados entre inquilinos.** A regeneração só reinsere eventos de linhas vivas
(`deleted_at is null`, `status = 'finished'`): histórico cuja origem foi apagada logicamente **não volta**.
- **Marcos de carreira perdidos permanentemente** - `source_key` é fixa e a conquista não é reconquistada.
- **Exaustão de recursos**: DELETE + INSERT de custo arbitrário sobre a tabela inteira, sem limite de taxa.

## Condições de exploração

Apenas uma conta autenticada. Sem papel especial, sem feature flag, sem configuração insegura.

## Sugestão de correção

Nova migration:

```sql
-- Nenhuma das duas é chamada pelo cliente: recalculate_player_career as invoca
-- internamente e, sendo security definer, não depende destes grants.
revoke execute on function public.regenerate_career_events_for_sessions(uuid[]) from authenticated;
revoke execute on function public.regenerate_player_milestones(uuid) from authenticated;

create or replace function public.recalculate_player_career(p_player_id uuid)
returns void
language plpgsql
security definer
set search_path = public
as $$
declare
affected uuid[];
begin
if (select auth.uid()) is null then
raise exception 'Not authenticated' using errcode = '42501';
end if;
if not public.current_user_is_player_admin(p_player_id) then
raise exception 'Not authorized to recalculate this player career' using errcode = '42501';
end if;
-- ... corpo atual ...
end;
$$;
```

## Critérios de aceite

- [ ] `regenerate_career_events_for_sessions(uuid[])` e `regenerate_player_milestones(uuid)` não tem mais `execute` para `authenticated` (conferir em `information_schema.role_routine_grants`).
- [ ] `recalculate_player_career(uuid)` levanta `42501` quando o chamador não é admin do jogador alvo.
- [ ] Suite `dbtest.ts` em `src/test/db/` cria duas contas em comunidades distintas e prova que a conta A não consegue alterar `career_events` da conta B por nenhuma das três RPCs.
- [ ] O fluxo legítimo de recálculo após claim de jogador continua verde.
- [ ] `npm run typecheck` e `npm run test` verdes.
```

RESOLVIDA · As tres RPCs perderam o execute de authenticated. A sugestão de guarda em recalculate_player_career **não foi seguida**: ela quebraria o cadastro com claim code, porque a função roda dentro do trigger de signup, onde auth.uid() ainda e NULL. Dois testes cobrem isso — a recusa das tres chamadas e o claim de jogador continuando a funcionar.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] reset_product_data apaga dados de todos os inquilinos apesar de receber uma conta alvo

**Labels:** `security`, `severity:média`, `database`, `data-loss`

## Problema

`public.reset_product_data(target_account_uuid text)` recebe um UUID de conta, mas apenas os dois últimos DELETE usam esse parâmetro. Os dezesseis anteriores varrem as tabelas inteiras, de todas as contas e comunidades.

## Por que é explorável

Não é um furo de privilégio — a função exige capability `reset_product_data` (hoje só `master`, após `20260729000000_reset_scaffold_programmer_revoke.sql:8`) mais `require_aal2()`. O problema é o raio de alcance: a assinatura e o chamador sugerem um reset por conta, e o efeito real é global e irreversível. `src/infra/supabase/resetScaffoldCloudService.ts:3` passa um único UUID de conta, reforçando a leitura errada.

## Evidência

`supabase/migrations/20260905185744_versioned_player_evaluation_source.sql:315-356`
```sql
create or replace function public.reset_product_data(target_account_uuid text)
returns void
language plpgsql
security definer set search_path = public
as $$
begin
if not public.has_capability('reset_product_data') then
raise exception 'Not authorized: missing reset_product_data capability';
end if;
perform public.require_aal2();
...
delete from public.point_events; -- linha 331, sem WHERE
delete from public.games; -- linha 332, sem WHERE
delete from public.teams; -- linha 333, sem WHERE
delete from public.sessions; -- linha 334, sem WHERE
delete from public.career_events; -- linha 339, sem WHERE
delete from public.player_evaluations; -- linha 340, sem WHERE
delete from public.self_evaluations; -- linha 341, sem WHERE
delete from public.community_players; -- linha 346, sem WHERE
...
delete from public.players -- linha 353, unico DELETE escapado
where owner_id = target_account_uuid::uuid
and not has_account_identity_history;
delete from public.communities where owner_id = target_account_uuid::uuid; -- linha 356
end;
$$;
```

## Impacto

Perda total e irreversível dos dados operacionais de toda a plataforma (sessões, jogos, pontos, avaliações, carreira, presenças, relatórios) a partir de uma ação que aparenta afetar apenas uma conta.

## Condições de exploração

Requer papel `master` autenticado com AAL2. Risco de erro operacional e de abuso por conta privilegiada comprometida, não de escalonamento.

## Sugestão de correção
```

Escopar cada DELETE pelo inquilino alvo, por exemplo:

```
```sql
delete from public.point_events
where session_id in (select id from public.sessions where owner_id = target_account_uuid::uuid);
delete from public.sessions where owner_id = target_account_uuid::uuid;
```
```

Se o escopo global for realmente desejado para um cenário de scaffold, renomear a função para explicitar isso (por exemplo `reset\_all\_product\_data()`), remover o parâmetro enganoso e exigir confirmação explícita.

Critérios de aceite

- [] Nenhum `delete from public.<tabela>;` sem cláusula de escopo permanece na função (ou o escopo global esta explícito no nome e sem parâmetro de conta).
- [] Suite `dbtest.ts` cria duas contas com dados, executa o reset da conta A e prova que os dados da conta B permanecem intactos.
- [] `src/infra/supabase/resetScaffoldCloudService.ts` e a UI de Gestão refletem o escopo real da operação.

RESOLVIDA · Os dezesseis DELETE incondicionais foram escopados pela conta alvo, alvo em branco passou a ser recusado e a função foi endurecida para `search_path = ''`. O teste cria duas contas com dados, reseta uma e prova que a outra sobreviveu inteira.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Política de INSERT em modification_logs com WITH CHECK (true) permite forjar trilha de auditoria

Labels: `security`, `severity:média`, `database`, `rls`, `audit`

Problema

A política de INSERT de `public.modification\_logs` aceita qualquer linha de qualquer usuário autenticado.

Por que é explorável

A política de leitura da mesma tabela é corretamente escopada (``20260610161203_backend_operational_sync.sql:572-576``):

```
```sql
create policy "Community members can read modification logs" on public.modification_logs
for select to authenticated
using (
 owner_id = (select auth.uid())
or (community_id is not null and public.current_user_has_community_role(community_id))
);
```
```

Mas `insert` avalia somente `with check`, que é `true`. Um usuário autenticado pode inserir registros de auditoria com `owner\_id`, `community\_id`, nome de tabela e payload arbitrários – atribuindo ações a terceiros:

```
```
POST /rest/v1/modification_logs
{ "owner_id": "<uuid de outro usuario>", "community_id": "<uuid alheio>", "table_name": "sessions", ... }
```
```

A política nasceu para destravar o trigger `log\_table\_changes()`, mas o trigger executa como dono da função e não depende de um grant tao amplo.

Evidência

```
`supabase/migrations/20260820110000_modification_logs_insert_policy.sql:7-10`
```sql
drop policy if exists "Authenticated users can insert modification logs" on public.modification_logs;
create policy "Authenticated users can insert modification logs" on public.modification_logs
for insert to authenticated
with check (true);
```
```

Impacto

Perda de integridade e de não-repúdio da trilha de auditoria: poluição do log, atribuição falsa de ações a outros usuários e mascaramento de atividade real durante uma investigação.

Condições de exploração

Conta autenticada. Sem pre-requisitos adicionais.

Sugestão de correção

```
```sql
drop policy if exists "Authenticated users can insert modification logs" on public.modification_logs;
create policy "Authenticated users can insert modification logs" on public.modification_logs
for insert to authenticated
with check (owner_id = (select auth.uid()));
```
```

Se o trigger precisar gravar linhas em nome de outro `owner\_id`, torna-lo `security definer` com `set search\_path = public` e remover por completo a política de INSERT do papel `authenticated`.

Critérios de aceite

- [] Um usuário autenticado não consegue inserir em `modification\_logs` uma linha com `owner\_id` diferente do seu.
- [] O trigger `log\_table\_changes()` continua gravando corretamente em update/upsert (sem 42501) – coberto por `dbtest.ts`.
- [] Teste que prove a negação do INSERT forjado.

RESOLVIDA · Corrigida pela causa e não pelo sintoma: o security definer foi restaurado em log_table_changes(), a policy permissiva removida e o insert revogado de authenticated. Os testes provam a recusa da linha forjada e que o trigger segue gravando auditoria sem policy nenhuma.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Políticas de UPDATE/DELETE preservam privilégio de membros rebaixados (ramo owner_id)

Labels: `security`, `severity:baixa`, `database`, `rls`

Problema

Nas tabelas `community\_rules`, `whatsapp\_list\_templates` e `community\_players`, o INSERT exige **as duas** condições (`AND`) enquanto UPDATE/DELETE aceitam **qualquer uma** (`OR`). Como `owner\_id` guarda quem criou a linha, o primeiro ramo permanece verdadeiro para sempre.

Por que é explorável

Um usuário que criou a linha enquanto era `owner`/`admin` e depois foi rebaixado a membro comum continua podendo alterar e apagar aquela linha, embora a interface já não ofereça a ação (`permissions.canEditRules` em `src/components/community/CommunitiesView.tsx:539`). O gate de papel existe no servidor, mas o ramo de posse o contorna – exatamente o caso em que a UI vira a única barreira restante.

Evidência

```
`supabase/migrations/20260610161203_backend_operational_sync.sql:542-551`
```sql
create policy "Community owners and admins can insert community rules" on public.community_rules
for insert to authenticated
with check (owner_id = (select auth.uid()) and public.current_user_has_community_role(community_id, array['owner', 'admin']));

create policy "Community owners and admins can update community rules" on public.community_rules
for update to authenticated
using (owner_id = (select auth.uid()) or public.current_user_has_community_role(community_id, array['owner', 'admin']))
with check (owner_id = (select auth.uid()) or public.current_user_has_community_role(community_id, array['owner', 'admin']));

create policy "Community owners and admins can delete community rules" on public.community_rules
for delete to authenticated
```

```
using (owner_id = (select auth.uid()) or public.current_user_has_community_role(community_id, array['owner', 'admin']));
```

```

Mesmo padrão em `20260610161203\_backend\_operational\_sync.sql:562-569` (`whatsapp\_list\_templates`) e `20260617180615\_community\_players\_optimization.sql:81-96` (`community\_players`).

Impacto

Privilégio residual após rebaixamento: edição e remoção de regras da comunidade, templates e vínculos de jogador por quem já perdeu o papel.

Condições de exploração

Requer que o usuário tenha criado a linha enquanto era owner/admin e depois tenha sido rebaixado sem sair da comunidade. Não afeta quem nunca teve o papel.

Sugestão de correção

Trocar o `or` por `and` nas políticas de UPDATE e DELETE das três tabelas, alinhando ao INSERT:

```
```sql
using (owner_id = (select auth.uid()) and public.current_user_has_community_role(community_id, array['owner', 'admin']))
```

```

Alternativamente, migrar essas escritas para RPCs semânticas, como já foi feito com `sessions` (`20260827210000\_target\_session\_root.sql`) e `community\_memberships`.

Critérios de aceite

- [] Teste `dbtest.ts`: usuário cria regra como admin, é rebaixado a `member`, e o UPDATE e o DELETE daquela linha passam a ser negados.
- [] Owner/admin atuais continuam podendo editar regras criadas por outros.
- [] Mesma cobertura para `whatsapp\_list\_templates` e `community\_players`.

RESOLVIDA · UPDATE e DELETE alinhados ao INSERT nas tres tabelas. O teste promove um usuário a admin, deixa que ele crie a linha, rebaixa a membro e prova que UPDATE e DELETE passam a não afetar linha alguma — e que a escrita legítima volta quando ele e repromovido.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Funções auxiliares expõem identidade e grafo de papéis a qualquer usuário autenticado

Labels: `security`, `severity:baixa`, `database`, `privacy`

Problema

Duas funções `security definer` concedidas a `authenticated` aceitam alvo arbitrário e contornam o modelo de privacidade construído nas migrações `20260726170000\_community\_profile\_privacy.sql` e `20260726210000\_profile\_visibility\_status\_rules.sql`.

Por que é explorável

`find\_player\_by\_username(text)` — consulta a tabela global `players` ignorando o RLS (`current\_user\_can\_access\_player`) e devolve nome real e UUID interno de qualquer atleta. Além da divulgação em si, fornece o `player\_id` que as RPCs de carreira consomem como entrada, encurtando a cadeia de exploração daquele achado.

`community\_capabilities(uuid, uuid)` — o parâmetro `target\_user\_id` é livre e não há comparação com `auth.uid()`. O wrapper seguro `current\_user\_has\_community\_capability` fixa o usuário corrente, mas a função subjacente também está concedida a `authenticated` e pode ser chamada diretamente com qualquer par (comunidade, usuário).

Evidência

```
supabase/migrations/20260610161256_global_athlete_identity.sql:63-78
```sql
create or replace function public.find_player_by_username(target_username text)
returns table (id uuid, username text, name text)
language sql

```

```

stable
security definer
set search_path = public
as $$
select p.id, p.username, p.name
from public.players p
where lower(p.username) = lower(trim(target_username))
and p.deleted_at is null
limit 1;
$$;

grant execute on function public.find_player_by_username(text) to authenticated;
```

`supabase/migrations/20260905185744_versioned_player_evaluation_source.sql:15-23` e o grant em
`supabase/migrations/20260827150000_normalize_governance_and_organizer.sql:171`
```sql
create or replace function public.community_capabilities(
target_community_id uuid,
target_user_id uuid
)
returns setof text
language sql
stable
security definer
set search_path = ''
...
grant execute on function public.community_capabilities(uuid, uuid) to authenticated;
```

## Impacto

Divulgação de identidade entre inquilinos e mapeamento do grafo de papéis e pertencimento da plataforma – útil para
escolher alvos de engenharia social e para encadear com os achados de carreira. Somente leitura.

## Condições de exploração

Conta autenticada. `find_player_by_username` busca por igualdade exata, o que limita enumeração em massa; ela existe
para o fluxo legítimo de vínculo de conta, então a correção deve preservar esse caso de uso.

## Sugestão de correção

```sql
-- Manter apenas o wrapper de usuario corrente exposto, como ja e feito com
-- assert_target_session_write_authorized (20260828034435:172-173).
revoke execute on function public.community_capabilities(uuid, uuid) from authenticated;

-- Enxugar o retorno ao minimo necessario para o fluxo de vínculo.
create or replace function public.find_player_by_username(target_username text)
returns table (id uuid, username text)
...
```

Adicionar limite de taxa por chamador em `find_player_by_username`.

## Critérios de aceite

- [ ] `community_capabilities(uuid, uuid)` não tem mais `execute` para `authenticated`;
`current_user_has_community_capability` continua funcionando em todas as políticas que o usam.
- [ ] `find_player_by_username` não devolve mais `name`, ou o devolve apenas quando o chamador compartilha comunidade com
o alvo.
- [ ] Fluxo de vínculo de jogador por username continua verde.
- [ ] `npm run typecheck` e `npm run test` verdes.

```

RESOLVIDA · `community_capabilities` foi revogada de `authenticated`. Em `find_player_by_username` a assinatura foi preservada e apenas `name` passou a ser condicional, porque remover o campo quebraria a busca de vínculo e remover a função quebraria a checagem de `username` no cadastro.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Bucket de avatares tem leitura anônima, expondo propostas não aprovadas

****Labels:**** `security`, `severity:baixa`, `storage`, `privacy`

Problema

A política de SELECT de `storage.objects` para o bucket `avatars` não declara cláusula `to`, portanto vale para o papel `public` — inclusive `anon`.

Por que é explorável

As três políticas de escrita logo abaixo são corretamente restritas a `to authenticated` + `current\_user\_is\_player\_admin`, mas a leitura cobre o bucket inteiro, incluindo o prefixo `proposals/<player\_id>/` — ou seja, fotos que ****ainda não passaram pela aprovação**** (ou que foram rejeitadas). O UUID de jogador que compõe o caminho é obtível via `find\_player\_by\_username`.

Evidência

```
`supabase/migrations/20260624133117_player_avatars_approval.sql:285-287`  
```sql
```

```
create policy "Avatars are publicly readable" on storage.objects
for select
using (bucket_id = 'avatars');
```
```

Contraste com a política de upload nas linhas 290-297, corretamente restrita:

```
```sql  
create policy "Player admins can upload avatar candidates" on storage.objects
for insert to authenticated
with check (
 bucket_id = 'avatars'
 and (storage.foldername(name))[1] = 'proposals'
 and public.current_user_is_player_admin(((storage.foldername(name))[2]))::uuid
);
```
```

Impacto

Imagens submetidas e pendentes ou rejeitadas de qualquer atleta ficam acessíveis sem autenticação a quem souber o caminho do objeto, esvaziando o propósito do fluxo de aprovação de avatar.

Condições de exploração

Exige conhecer o caminho do objeto, que contém o UUID do jogador. Avatares ****já aprovados**** são públicos por design — o problema é restrito ao prefixo `proposals/`.

Sugestão de correção

```
```sql  
drop policy if exists "Avatars are publicly readable" on storage.objects;

create policy "Approved avatars are publicly readable" on storage.objects
for select
using (bucket_id = 'avatars' and (storage.foldername(name))[1] <> 'proposals');

create policy "Player admins can read avatar candidates" on storage.objects
for select to authenticated
using (
 bucket_id = 'avatars'
 and (storage.foldername(name))[1] = 'proposals'
 and public.current_user_is_player_admin(((storage.foldername(name))[2]))::uuid
);
```
```

Critérios de aceite

- [] Requisição anônima a um objeto sob `proposals/` retorna 403/404.
- [] Requisição anônima a um avatar aprovado continua retornando 200.
- [] Admin do jogador continua conseguindo listar e visualizar as propostas daquele jogador na caixa de aprovação (`src/components/player/AvatarApprovalInbox.tsx`).

RESOLVIDA · Leitura pública restrita ao que não esta sob proposals/, com segunda policy dando esse prefixo ao admin do jogador. O teste liga o RLS no stand-in de storage do harness dentro de uma transação revertida — sem isso as policies existiriam sem nunca serem avaliadas, e o teste não provaria nada.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

```
# [Segurança] Adicionar Content-Security-Policy ao nginx

**Labels:** `security`, `severity:baixa`, `hardening`, `deploy`

## Problema

Não há nenhum header `Content-Security-Policy` em `nginx.conf`, `index.html` ou `vite.config.ts`.

## Por que importa

**Este item não decorre de nenhuma falha de XSS existente.** A varredura da categoria encontrou zero sinks:
sem `dangerouslySetInnerHTML`, `eval`, `new Function`, `srcdoc`, `outerHTML`, `insertAdjacentHTML` ou
`document.write` em `src/`; sem renderização de markdown; sem `href` dinâmico. O código está limpo hoje.

O ponto é a ausência de rede de contenção: o `X-XSS-Protection` presente é obsoleto e ignorado por navegadores
modernos, então uma regressão futura ou o comprometimento de uma dependência teria execução livre.

## Evidência

`nginx.conf:25-33` — os três headers presentes, sem CSP em nenhum bloco:
```nginx
add_header X-Frame-Options "SAMEORIGIN";
add_header X-XSS-Protection "1; mode=block";
add_header X-Content-Type-Options "nosniff";
}

Security headers (valem para o location / e demais sem add_header proprio)
add_header X-Frame-Options "SAMEORIGIN";
add_header X-XSS-Protection "1; mode=block";
add_header X-Content-Type-Options "nosniff";
}
```

## Impacto

Sem impacto direto no estado atual do código. Endurecimento preventivo contra regressão e contra comprometimento da
cadeia de suprimentos.

## Sugestão de correção

Adicionar em ambos os blocos (o comentário das linhas 22-24 já documenta por que o header precisa ser repetido no
`location` de assets):

```nginx
add_header Content-Security-Policy "default-src 'self'; connect-src 'self' https://*.supabase.co; img-src 'self' data:
blob: https://*.supabase.co; style-src 'self' 'unsafe-inline'; font-src 'self' data:; frame-src
https://challenges.cloudflare.com; script-src 'self' https://challenges.cloudflare.com; object-src 'none'; base-uri
'self'; form-action 'self'" always;
```

Validar em homologação antes de promover: o Turnstile (`@marsidev/react-turnstile`) precisa de
`challenges.cloudflare.com` em `script-src` e `frame-src`, e o Supabase Storage precisa constar em `img-src`.

## Critérios de aceite

- [ ] `curl -I` na aplicação publicada retorna `Content-Security-Policy` tanto em `/` quanto em um asset `.js`.
- [ ] Login, cadastro e recuperação de senha com Turnstile funcionam sem violação de CSP no console.
- [ ] Upload e exibição de avatar funcionam sem violação de CSP.
- [ ] Nenhum erro de CSP no console navegando pelas telas principais.
```

RESOLVIDA · CSP publicado nos dois blocos do `nginx.conf`. Verificado servindo o bundle real pelo nginx real: header presente em / e nos assets, app carregado no navegador e Web Worker do balanceador executado, sem violação originada pela aplicação. Turnstile e Supabase seguem pendentes de conferência em homologacao, por dependerem de credencial.

--- FIM ISSUE 7 ---